Pthreads

- Specification part of larger IEEE POSIX standard
  - POSIX is the **P**ortable **O**perating **S**ystem **I**nterface
  - Standard C API for threading libraries
    - IBM provides Fortran API
  - Introduced in 1995
- Explicit threading of application
  - User calls functions to create/destroy threads

Materials from http://www.llnl.gov/computing/tutorials/pthreads/



The Pthreads Model

- Execution Model
  - Explicit parallelism
  - Explicit synchronization
- Productivity
  - Not a primary objective
  - Library for existing language
  - Low level of abstraction
  - Uses opaque objects – prevents expansion



The Pthreads Model

- Performance
  - No attempts to manage latency
  - Load balancing left to OS
  - Developer responsible for creating high degree of parallelism by spawning threads
- Portability
  - Library widely available



Pthreads Thread Management

- User creates/terminates threads
- Thread creation
  - `pthread_create`
  - Accepts a single argument (void *)
- Thread termination
  - `pthread_exit`
  - Called from within terminating thread



Pthreads Synchronization

Mutual Exclusion Variables (mutexes)

- Mutexes must be initialized before use
- Attribute object can be initialized to enable error checking

```
pthread_mutex_t mutexsum;
void *dot_product(void *arg) {
  …
  pthread_mutex_lock (&mutexsum);
  sum += mysum;
  pthread_mutex_unlock (&mutexsum);
  …
}
int main() {
  pthread_mutex_init(&mutexsum, NULL);
  …
  pthread_mutex_destroy(&mutexsum);
}
```



Pthreads Synchronization

Condition Variables

- Allows threads to synchronize based on value of data
- Threads avoid continuous polling to check condition
- Always used in conjunction with a mutex
  - Waiting thread(s) obtain mutex then wait
    - pthread_cond_wait() function unlocks mutex
    - mutex locked for thread when it is awakened by signal
  - Signaling thread obtains lock then issues signal
    - pthread_cond_signal() releases mutex



Condition Variable Example

Two threads update a counter
Third thread waits until counter reaches a threshold

```
pthread_mutex_t mtx;
pthread_cond_t cv;

int main() {
…
pthread_mutex_init(&mtx, NULL);
pthread_cond_init (&cv, NULL);
…
pthread_create(&threads[0], &attr,
               inc_count, (void *)&thread_ids[0]);
pthread_create(&threads[1], &attr,
               inc_count, (void *)&thread_ids[1]);
pthread_create(&threads[2], &attr,
               watch_count, (void *)&thread_ids[2]);
…
}
```



Condition Variable Example

Incrementing Threads

```
void *inc_count(void *idp) {
…
for (i=0; i<TCOUNT; ++i) {
  pthread_mutex_lock(&mtx);
  ++count;
  if (count == LIMIT)
    pthread_cond_signal(&cv);
  pthread_mutex_unlock(&mtx);
  …
}
…
}
```

Waiting Thread

```
void *watch_count(void *idp) {
…
pthread_mutex_lock(&mtx);
while (count < COUNT_LIMIT) {
  pthread_cond_wait(&cv, &mtx);
}
pthread_mutex_unlock(&mtx);
…
}
```

pthread_cond_broadcast() used if multiple threads waiting on signal



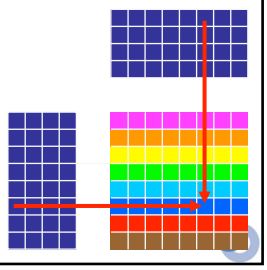
PThreads Matrix Multiply

```
int tids[M];
pthread_t threads[M];
pthread_attr_t attr;
pthread_attr_init(&attr);
pthread_attr_setdetachstate(
  &attr,
  PTHREAD_CREATE_JOINABLE);

for (i=0; i<M; ++i) {
  tids[i] = i;
  pthread_create(&threads[i],
   &attr, work, (void *) &tids[i]);
}

for (i=0; i<M; ++i) {
  pthread_join(threads[i], NULL);
}
```

```
void* work(void* tid) {
  for(int j=0; j<N; ++j) {
    for(int k=0; k<L; ++k) {
      C[tid][j] +=
        A[tid][k]*B[k][j];
    }
  }
  pthread_exit(NULL);
}
```



References

OpenMP

http://www.openmp.org

https://hpc-tutorials.llnl.gov/openmp/

PThreads

https://hpc-tutorials.llnl.gov/posix/

"Pthreads Programming". B. Nichols et al. O'Reilly and Associates.

"Programming With POSIX Threads". D. Butenhof. Addison Wesley

Message Passing Model

- Large scale parallelism (up to 100k+ CPUs)
- Multiple (possibly heterogeneous) system images
- Distributed memory
  - Nodes == servers can only access local data
  - Application (User) responsible for:
    - Distributing data
    - Redistributing data (when necessary)
    - Maintaining memory coherent



Message Passing Model

- Explicit communication
  - Two-sided P2P:
    - Communication initiated on one side requires matching action on the remote side
    - E.g. MPI_Send – MPI_Recv
  - One-sided P2P:
    - Communication is initiated on one side and no action is required on the other
    - E.g. MPI_Get/Put, gasnet_get/put ...
- Implicit synchronization with two-sided communication
  - The matching of communication operations from both sides ensures synchronization



Message Passing Model

- Objectives of the model
  - Enabling parallelization on highly scalable hardware
  - Support for heterogeneous systems
  - Often coarse-grained parallelism
- Main issues
  - Communication
  - Synchronization
  - Load balancing



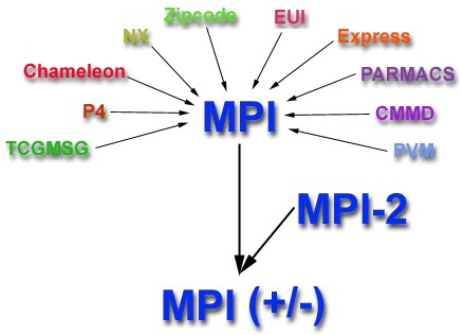
Projects of Interest

- **Message Passing Interface (MPI)**
  - De facto standard for this model
  - Deemed low level and difficult to program
  - Two-sided and one-sided communication
- Charm++ (PP@Master)
  - Asynchronous Remote Method Invocation (RMI) communication
  - Split-phase programming model
    - No synchronous communication
    - Caller provides a callback handler to asynchronously process "return" value



Message Passing Interface (MPI)

- 1980s – early 1990s
  - Distributed memory, parallel computing develops
  - Many incompatible software tools
  - Usually tradeoffs between portability, performance, functionality and price
- Recognition of the need for a standard arose.



Material from: https://hpc-tutorials.llnl.gov/mpi/



Message Passing Interface (MPI)

- Standard based on the consensus of the MPI Forum
  - Not sanctioned by any major standards body
  - Wide practical acceptance
  - No effective alternative to date
- First draft of the MPI-1 standard presented at Supercomputing 1993
- Current standard MPI-2 developed between 1995 and 1997
- Standardization committee open to all members of the HPC community

Further reading and standard documents: http://www.mpi-forum.org/



Message Passing Interface (MPI)

- Objectives
  - High performance and scalability
  - Portability
  - Productivity was not an objective (it became one)
- Used as communication layer for higher-level libraries
  - Often for more productivity-oriented libraries
  - ISO/OSI layer-5 interface
    - Communication is reliable and sessions are managed internally
    - Data is not structured



MPI: Specification, not Implementation

- Language Independent Specification (LIS)
- Library implementations of MPI vary in:
  - Performance
    - Target or rely on specific hardware (RDMA, PIM, Coprocessors …)
    - Provide load-balancing and processor virtualization (e.g., AMPI)
  - Functionality
    - Support for parallel I/O
    - Support for multithreading within MPI processes
- Standard provides language bindings for Fortran, C and C++
- Implementations usually provide APIs for C, C++ and Fortran
- Project implementations for Python, OCaml, and Java



MPI – Programming Model

Execution Model

- Explicitly parallel
  - Programmer responsible for correctly identifying parallelism and for implementing parallel algorithms using MPI constructs
  - Multi-threaded view
- SPMD
- Explicit data distribution
- Flat parallelism
  - Number of tasks dedicated to run a parallel program is static
- Processor Consistency (one-sided communication)



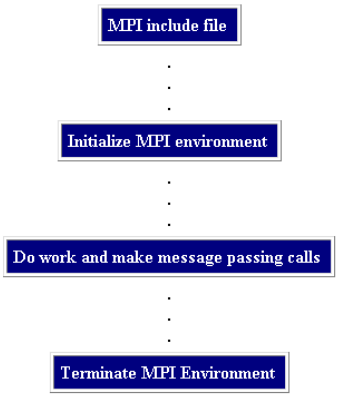
MPI – Programming Model

- Productivity
  - Not a principal objective
    - Low level of abstraction
    - Communication is not structured (marshalling done by the user)
- Performance
  - Vendor implementations exploit native hardware features to optimize performance
- Portability
  - Most vendors provide an implementation
    - E.g., Specialized open source versions of MPICH, LAM or OpenMPI
  - ”Standard” ensures compatibility



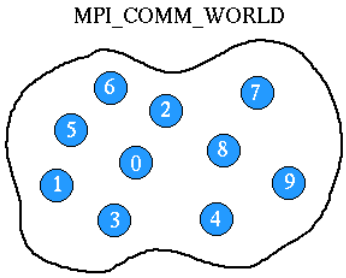
MPI – Program Structure

General program structure



Communicators and groups

- Collection of processes that may communicate
- Unique rank (processor ID) within communicator
- Default communicator: MPI_COMM_WORLD

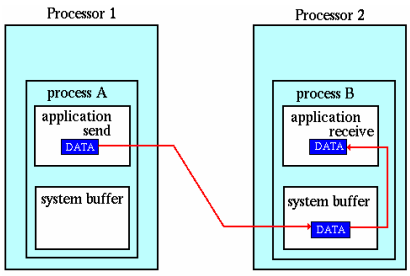




MPI – Point to Point Communication

Buffering when tasks are out of sync

- If a receive operation is not ready, sent data is buffered
  - On receiving side, sending side or both
- User can manage buffering memory on sending side



Path of a message buffered at the receiving process



MPI – Point to Point Communication

Types of Point-to-Point Operations:

- Message passing between two, and only two, different MPI tasks
  - One task performs a send operation
  - The other task matches with a receive operation
- Different types of send/receive routines used for different purposes
  - Synchronous send
  - Blocking send / blocking receive
  - Non-blocking send / non-blocking receive
  - Buffered send
  - Combined send/receive
  - "Ready" send
- Any type of send can be paired with any type of receive
- Test and Probe routines to check the status of pending operations



MPI – Point to Point Communication

Blocking vs. Non-blocking

- Most routines can be used in either blocking or non-blocking mode
- Blocking communication routines
  - Blocking send routines only return when it is safe to reuse send buffer
    - Modifications to send buffer will not affect data received on the remote side
      - Data already sent
      - Data buffered in a system buffer
  - Blocking send calls can be synchronous
    - Handshaking with the receiver
  - Blocking send calls can be asynchronous
    - System buffer used to hold the data for eventual delivery to the receiver
  - Blocking receive calls only return after the data has arrived and is ready for use by the program



MPI – Point to Point Communication

Blocking communication example

```
#include "mpi.h"
#include <stdio.h>

int main(int argc,char *argv[]) {
  int numtasks, rank, dest, source, rc, count, tag=1;
  char inmsg, outmsg='x';
  MPI_Status Stat;

  MPI_Init(&argc,&argv);
  MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);

  if (rank == 0) {
    dest = 1;
    source = 1;
    rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest, tag, MPI_COMM_WORLD);
    rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source, tag, MPI_COMM_WORLD, &Stat);
  }
  else if (rank == 1) {
    dest = 0;
    source = 0;
    rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source, tag, MPI_COMM_WORLD, &Stat);
    rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest, tag, MPI_COMM_WORLD);
  }

  rc = MPI_Get_count(&Stat, MPI_CHAR, &count);
  printf("Task %d: Received %d char(s) from task %d with tag %d \n",
         rank, count, Stat.MPI_SOURCE, Stat.MPI_TAG);

  MPI_Finalize();
}
```



MPI – Point to Point Communication

Blocking vs. Non-blocking

- Non-blocking communication routines
  - Send and receive routines behave similarly
    - Return almost immediately
    - Do not wait for any communication events to complete
      - Message copying from user memory to system buffer space
      - Actual arrival of message
  - Operations "request" the MPI library to perform an operation
    - Operation is performed when its requirements are met (e.g., message arrives)
    - User cannot predict when that will happen
  - Unsafe to modify the application buffer until completion of operation
    - Wait and Test routines used to determine completion
- Non-blocking communications primarily used to overlap computation with communication and exploit possible performance gains



MPI – Point to Point Communication

Non-blocking communication example

```
MPI_Request reqs[4];
MPI_Status stats[4];

prev = rank-1;
next = rank+1;
if (rank == 0) prev = numtasks - 1;
if (rank == (numtasks - 1)) next = 0;

MPI_Irecv(&buf[0], 1, MPI_INT, prev, tag1, MPI_COMM_WORLD, &reqs[0]);
MPI_Irecv(&buf[1], 1, MPI_INT, next, tag2, MPI_COMM_WORLD, &reqs[1]);

MPI_Isend(&rank, 1, MPI_INT, prev, tag2, MPI_COMM_WORLD, &reqs[2]);
MPI_Isend(&rank, 1, MPI_INT, next, tag1, MPI_COMM_WORLD, &reqs[3]);

  {
    // do some work
    // work will overlap with previous communication
  }

MPI_Waitall(4, reqs, stats);
```



MPI – Point to Point Communication

Order and Fairness

- Message Ordering
  - Messages do not overtake each other
    - If a sender sends two messages (Message 1 and Message 2) in succession to the same destination, and both match the same receive, the receive operation will receive Message 1 before Message 2.
    - If a receiver posts two receives (Receive 1 and Receive 2), in succession, and both match the same message, Receive 1 will receive the message before Receive 2.
  - Ordering is not thread-safe
    - If multiple threads participate in the communication, no order is guaranteed
- Fairness of Message Delivery
  - No fairness guarantee
    - Programmer responsible for preventing operation starvation
  - Example: task 0 sends a message to task 2. However, task 1 sends a competing message that matches task 2's receive. Only one of the sends will complete.



MPI – Collective Communication

- All or None
  - Must involve **all** processes in the scope of the used communicator
  - User responsible to ensure all processes within a communicator participate in any collective operation
- Types of Collective Operations
  - Synchronization (barrier)
    - Processes wait until all members of the group reach the synchronization point
  - Data Movement
    - Broadcast, scatter/gather, all to all
  - Collective Computation (reductions)
    - One member of the group collects data from the other members and performs an operation (e.g., min, max, add, multiply, etc.) on that data



MPI – Collective Communication

Programming Considerations and Restrictions

- Collective operations are blocking
- Collective communication routines do not take message tag arguments
- Collective operations within subsets of processes
  - Partition the subsets into new groups
  - Attach the new groups to new communicators
- Can only be used with MPI predefined data types
  - Not with MPI Derived Data Types



MPI – Matrix Multiply (master task)

- Initialization

Common to both master and worker processes

```
#define NRA 15     // Number of rows in matrix A
#define NCA 25     // Number of columns in A
#define NCB 10     // Number of columns in B
#define TAG 0      // MPI communication tag
// Data structures
double A[NRA][NCA];// matrix A to be multiplied
double B[NCA][NCB];// matrix B to be multiplied
double C[NRA][NCB];// result matrix C
```

- Distribute data to workers

```
avgNumRows     = NRA/numWorkers;
remainingRows = NRA%numWorkers;
offset         = 0;
for (dest = 1; dest <= numWorkers; ++dest) {
  rows = (dest <= remainingRows) ? avgNumRows + 1 : avgNumRows;
  MPI_Send(&offset, 1, MPI_INT, dest, TAG, MPI_COMM_WORLD);
  MPI_Send(&rows, 1, MPI_INT, dest, TAG, MPI_COMM_WORLD);
  count = rows * NCA;
  // Send horizontal slice of A
  MPI_Send(&A[offset][0], count, MPI_DOUBLE, dest, TAG, MPI_COMM_WORLD);
  // Send matrix B
  count = NCA * NCB;
  MPI_Send(&B, count, MPI_DOUBLE, dest, TAG, MPI_COMM_WORLD);
  offset += rows;
}
```

- Wait for results from workers

```
for (i = 1; i <= numworkers; ++i)       {
  source = i;
  MPI_Recv(&offset, 1, MPI_INT, source, TAG, MPI_COMM_WORLD, &status);
  MPI_Recv(&rows, 1, MPI_INT, source, TAG, MPI_COMM_WORLD, &status);
  count = rows * NCB;
  MPI_Recv(&C[offset][0], count, MPI_DOUBLE, source, TAG,MPI_COMM_WORLD,&status);
}
```

280

MPI – Matrix Multiply (worker task)

- Receive data from master

```
source = 0;
MPI_Recv(&offset, 1, MPI_INT, source, TAG, MPI_COMM_WORLD, &status);
MPI_Recv(&rows,   1, MPI_INT, source, TAG, MPI_COMM_WORLD, &status);
// Receive horizontal slice of A
count = rows * NCA;
MPI_Recv(&A, count, MPI_DOUBLE, source, TAG, MPI_COMM_WORLD, &status);
// Receive matrix B
count = NCA * NCB;
MPI_Recv(&B, count, MPI_DOUBLE, source, TAG, MPI_COMM_WORLD, &status);
```

- Process data

```
// Compute the usual matrix multiplication on the slice of matrix A and matrix B
for (k = 0; k < NCB; ++k) {
  for (i = 0; i < rows; ++i) {
    C[i][k] = 0.0;
    for (j = 0; j < NCA; ++j) {
      C[i][k] += A[i][j] * B[j][k];
    }
  }
}
```

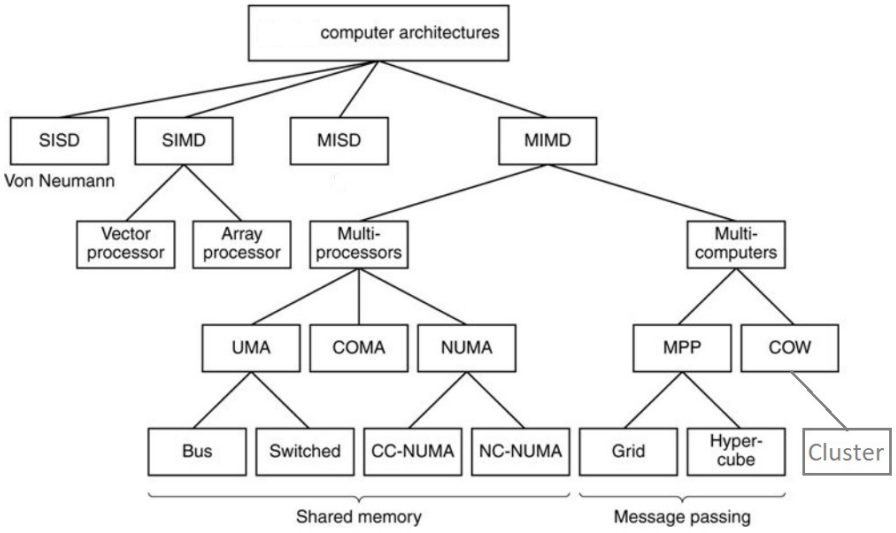
- Send results back to master

```
destination = 0;
MPI_Send(&offset, 1, MPI_INT, destination, TAG, MPI_COMM_WORLD);
MPI_Send(&rows, 1, MPI_INT, destination, TAG, MPI_COMM_WORLD);
count = rows * NCB;
// Send horizontal slice of result matrix C computed on this node
MPI_Send(&C, count, MPI_DOUBLE, destination, TAG, MPI_COMM_WORLD);
```



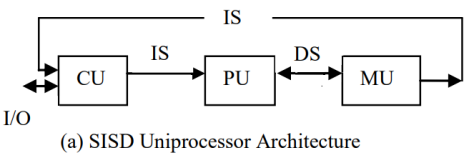
281

Arhitecturi Sisteme de Calcul

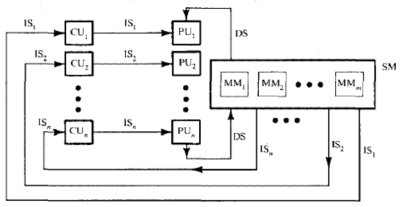


Taxonomii – Flynn

(c) MISD Architecture (the Systolic Array)

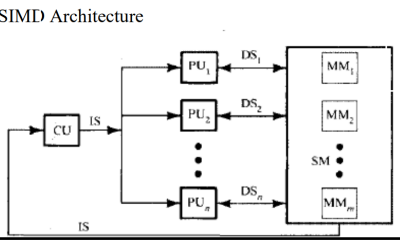


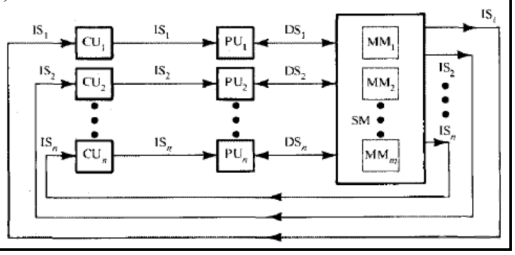
(a) SISD Uniprocessor Architecture



(d)MIMD architecture

(b) SIMD Architecture





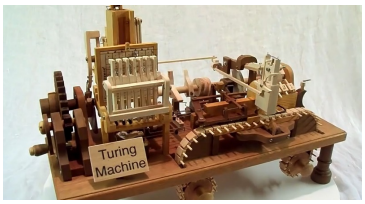
Modele ale calculului paralel

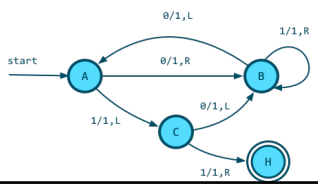
- Model RAM (Random Acces Machine)
- Modelul Pipeline
- Procesoare vectoriala
- Multiprocesoare cu memorie partajata
- Multiprocesoare cu transfer prin mesaje
- Modele PRAM (Parallel Random Acces Machine)
- Procesare Sistolica
- Procesare Data Flow

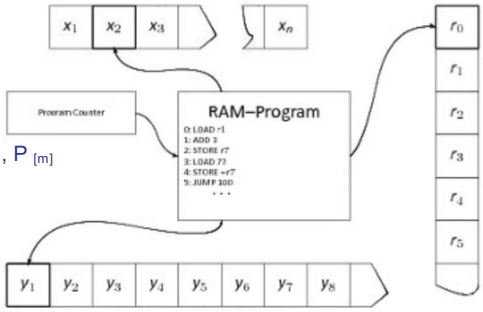


Model RAM (Random Acces Machine)

- O mașină cu acces aleatoriu (RAM):
  - Bandă intrare infinită X cu citire i ∈ N,
  - Bandă ieșire infinită Y cu scriere o ∈ N,
  - Registru contor de program PC
  - Program cu număr finit instrucțiuni P [1], …, P [m]
  - Memorie cu registrii r_0, r_1, r_2







- RAM = (X ,Y, S, s0, f, Z)
- S multimea finita a starilor automatului
- s_0 starea initiala
- X alfabetul finit al benzii de intrare
- Y alfabetul finit al benzii de iesire
- f functia de tranzitie, eventual partial definite

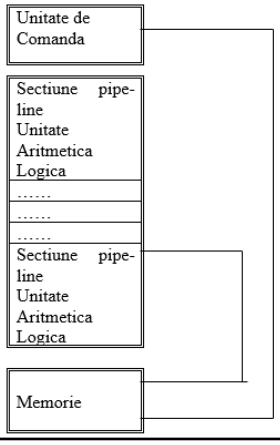
  $f : S \times X \rightarrow S \times Y$
- Z inclus in S – multime a starilor finale



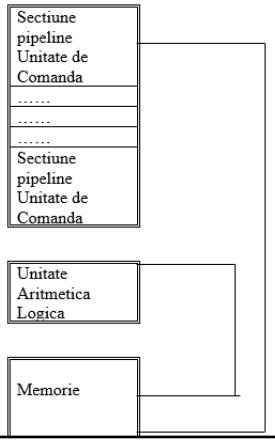
Modelul Pipeline

- Conceptul de pipeline consta in impartirea unui task T in subtask-uri T_1 , T_2 , ...,T_k si de a le atribui unor elemente de procesare.
- Paralelismul se poate realiza fie la nivel de:

prelucrare aritmetica



citire, interpretare, executie instructiuni

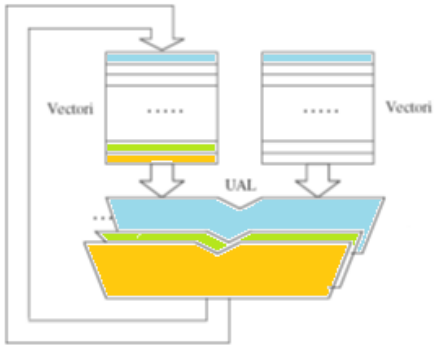




286

Procesoare vectoriala

- Set de instructiuni ce opereaza pe vectori
- Elemente de procesare vectoriala
- Disponibil pe sisteme monoprocesor ca extensii de procesare pe vectori
  - SSE, AVX, AltiVec, etc.



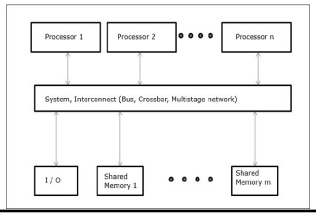


287

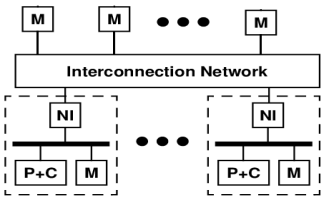
Multiprocesoare cu memorie partajata

- Fiecare procesor contine
  - Propria unitate de prelucrare
  - Propria memorie
  - Propria unitate de comanda
  - Comunica prin intermediul unei retele de comutare
- Fiecare procesor executa propriul set de instructiuni din memoria locala sau cea partajata
- Reteaua de comutare permite schimbul de informatii intre procesoare sau intre procesoare si memoria partajata

UMA



NUMA

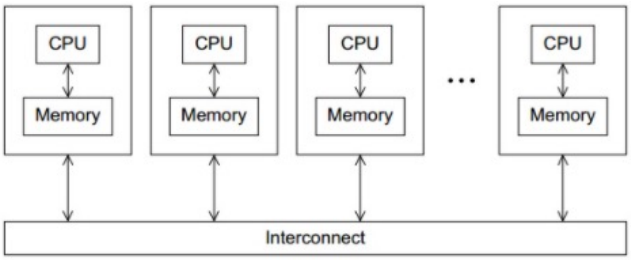




288

Multiprocesoare cu transfer prin mesaje

- Memoria este distribuita fiecarui procesor pentru cod si date
- Comunicarea datelor
  - Se face prin schimburi de mesaje
  - Printr-o retea de comutatie
  - Nu prin DMA/comutare de circuite

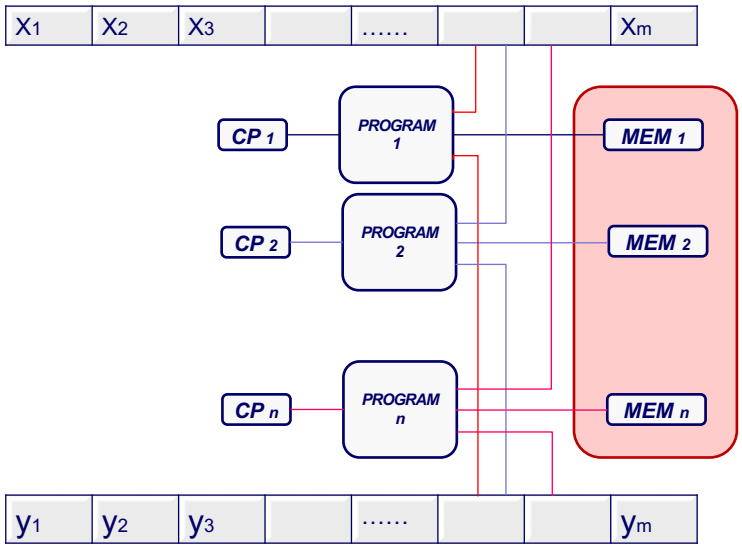




289

Modele PRAM – Parallel Random Acces Machine

290





290

Modele PRAM (2)

291

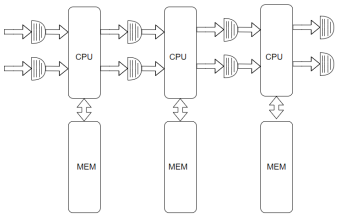
- Modele abstracte folosite pentru designul si analiza algoritmilor paraleli
- Set de procesoare ce au acces la o memorie partajata
- Procesoarele pot opera simultan si independent
- Procesoarele se pot coordona prin R/W la memorie
- Utile pentru designul algoritmilor paraleli deoarece permit un nivel crescut de paralelism
- Algoritmii PRAM sunt insa greu de mapat pe sisteme reale
- Tipuri de arhitecturi PRAM:
  - Exclusive Read Exclusive Write (EREW) – fiecare procesor poate doar citi sau scrie la un moment de timp – fara data-race-uri
  - Concurrent Read Exclusive Write (CREW) – mai multe procesoare citesc dintr-o memorie, doar un procesor scrie intr-o memorie
  - Priority PRAMs – procesoarele au prioritati la R/W

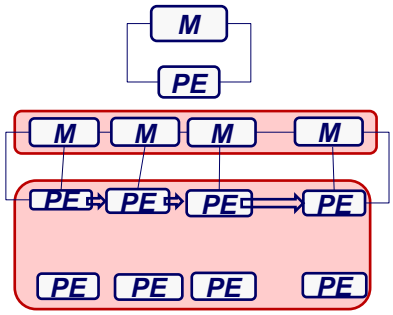


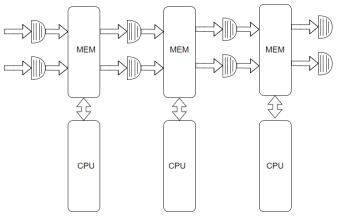
291

Procesare Sistolica

- Elemente de procesare identice aranjate intr-o structura pipeline
- Interconexiuni simple si regulate
  - Permite integrarea VLSI









292

Caracteristici ale Sistemelor Sistolice

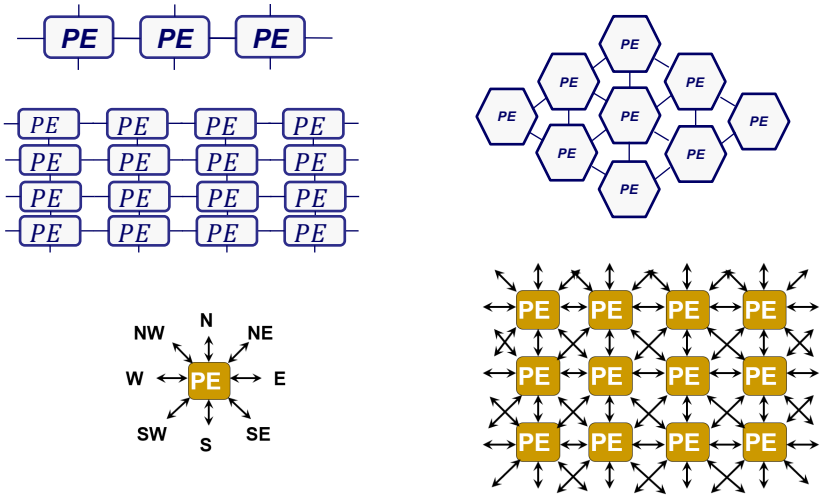
- Regularitate:
  - PE-uri identice ce permit scalabilitate crescuta
- Pipeline:
  - Calculele sunt asezate in banda de asamblare
  - Paralelismul crescut poate fi implementat eficient
- Localitate:
  - Datele sunt mutate doar adiacent
  - Comunicarea se face doar cu vecinii – overhead minimal
- Scalabilitate:
  - Sistemele de acest tip scaleaza usor prin adaugarea de noi PE-uri.
  - Se potrivesc pentru aplicatii ce necesita multa putere de calcul



293

294

Modalitati de conectare Sistolica



294

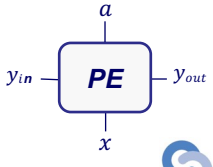
295

Exemplu de utilizare

- Fie
- A=$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{n1} & an_2 & \cdots & a_{nn} \end{bmatrix}$ Y=$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ X=$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$ Y_0=$\begin{bmatrix} y_{01} \\ y_{02} \\ \vdots \\ y_{0n} \end{bmatrix}$
- Dorim sa calculam
- Y=A*X+Y0
- Presupunem ca avem un element de procesare sistólica

Y	A					X	Y_0
y_1	a_{11}	a_{12}	a_{13}		a_{1n}	x_1	y_{01}
y_2	a_{21}	a_{22}	a_{23}		a_{2n}	x_2	y_{02}
y_3	a_{31}	a_{32}	a_{33}		a_{3n}	x_3	y_{03}
y_n	a_{n1}	a_{n2}	a_{n3}		a_{nn}	x_n	y_{0n}

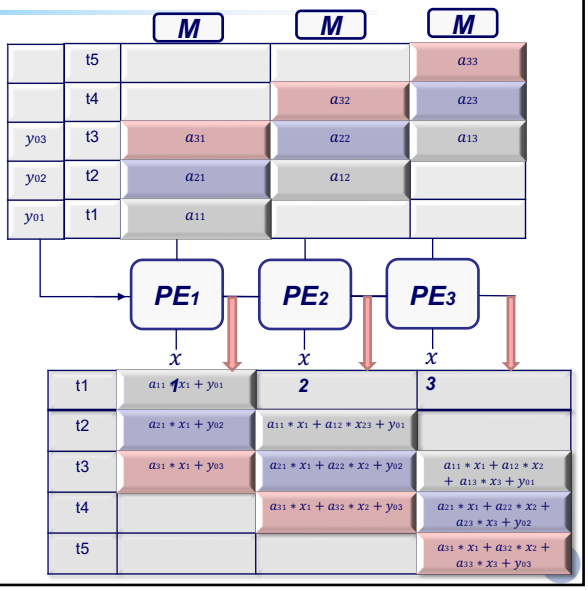
$y_{out} = a * x + yin$



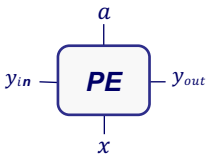
295

Exemplu unidimensional cu n=3

- $y_i = \sum_{j=1}^{n} a_{ij} * xj + y_{0i}$
- $y_1 = a_{11} * x_1 + a_{12} * x_2 + a_{13} * x_3 + y_{01}$
- $y_2 = a_{21} * x_1 + a_{22} * x_2 + a_{23} * x_3 + y_{02}$
- $y_3 = a_{31} * x_1 + a_{32} * x_2 + a_{33} * x_3 + y_{03}$



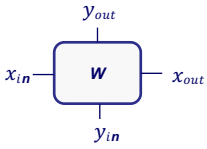
$y_{out} = a * x + yin$

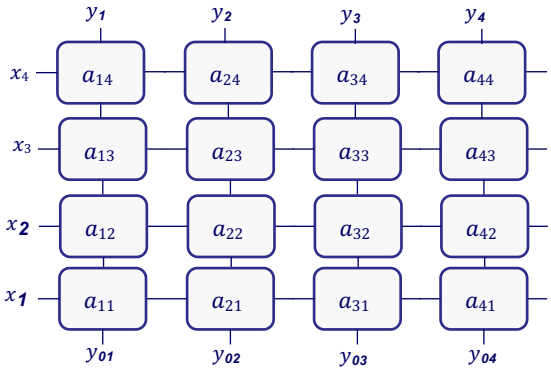


296

Exemplu bidimensional cu n=4

- $y_i = \sum_{j=1}^{n} a_{ij} * xj + y_{0i}$
- $y_1 = a_{11} * x_1 + a_{12} * x_2 + a_{13} * x_3 + a_{14} * x_4 + y_{01}$
- $y_2 = a_{21} * x_1 + a_{22} * x_2 + a_{23} * x_3 + a_{24} * x_4 + y_{02}$
- $y_3 = a_{31} * x_1 + a_{32} * x_2 + a_{33} * x_3 + a_{34} * x_4 + y_{03}$
- $y_4 = a_{41} * x_1 + a_{42} * x_2 + a_{43} * x_3 + a_{44} * x_4 + y_{04}$





$y_{out} = w * xin + yin$

$x_{out} = xin$

297

298

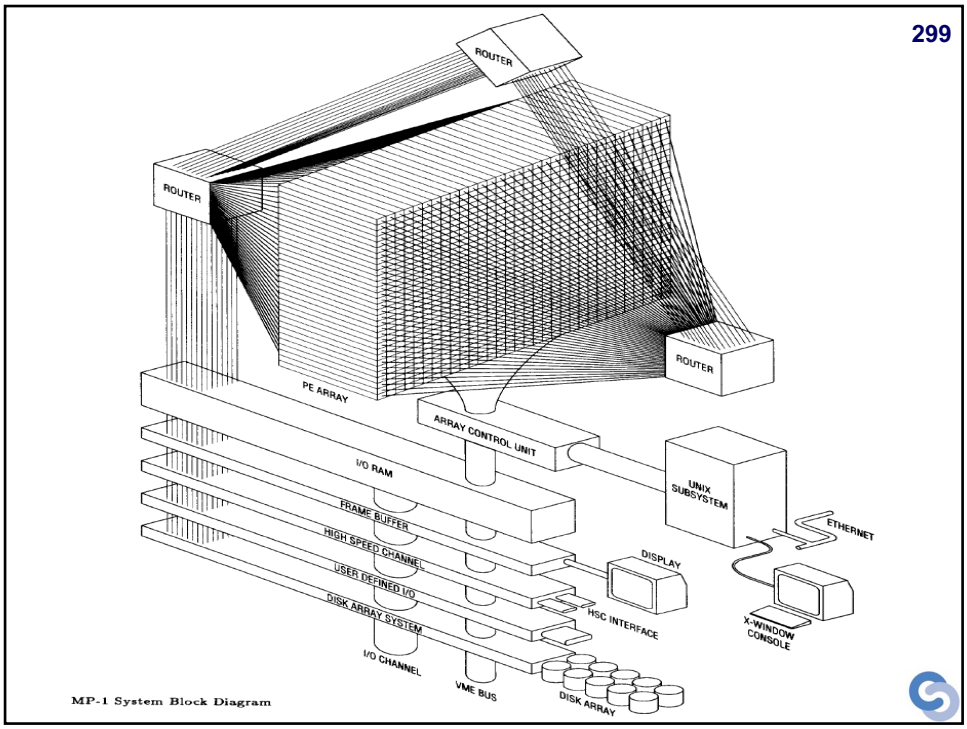
Algoritmi potriviti pentru Sisteme Sistolice

- Embarrassingly Parallel Algorithms: “trivial” paraleli
  - Fiecare PE actioneaza independent
  - Exemple:
    - Calcularea valorii de prag
    - Apartenenta unui element la un domeniu
- Near Embarrassingly Parallel Algorithms
  - Un PE are nevoie de informatii pe care le primeste de la alte PE-uri
  - Exemple:
    - Tratarea imaginilor
    - Detectia contururilor
    - Procesarea de semnale



298

299



299

300

Procesare Data Flow

- Este un mode bazat pe fluxul de date (Data Flow)
  - Execuția unei instructiuni de prelucarea este efectuata doar daca este disponibil operandul
  - Nu dispune de PC – activarea se face daca sunt disponibili toti operanzii
  - O instrucțiune este declanșată / executată când devine activă
  - Efectul declanșării unei instrucțiuni este prelucrarea datelor sale de intrare (operanzi) & generarea datelor de ieșire (rezultate)
  - Este o reprezentare grafica a modului in care circula informatia in sistem
  - Sunt utilizate pentru designul si analiza sistemelor de procesare a informatiilor



300

Caracteristici ale Sistemelor Data Flow

301

- Procese
  - Unitatile fundamentale de lucru intr-un system Data Flow
  - Reprezinta activitatile care transforma date dintr-o forma in alta
- Stocarea de date
  - Locul unde se stocheaza datele intre procese
- Data Flows
  - Reprezinta miscarea datelor intre procese si stocare
- Sistemele de Control
  - Sunt folosite pentru a determina flow-ul datelor in system
- Entitati externe
  - Reprezinta sursele si destinatiile datelor din sistem

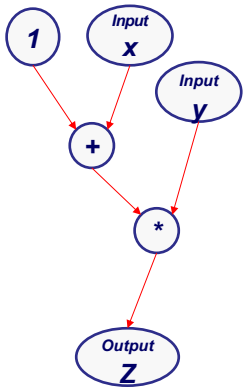


301

Exemplul de cod 1

- Un program Data Flow este compilat într-un graf Data Flow
  - Graf direcționat format din noduri
  - Reprezintă instrucțiuni și arce
  - Reprezintă dependențe de date între instrucțiuni
- Graficul Data Flow e similar cu un graf de dependență utilizat în reprezentările intermediare ale compilatoarelor
- În executarea programului datele se propagă de-a lungul arcelor în pachete de date, numite token-uri
- Acest flux de token-uri permite ca unele dintre noduri (instrucțiuni) sa inceapa executia și le declanșează

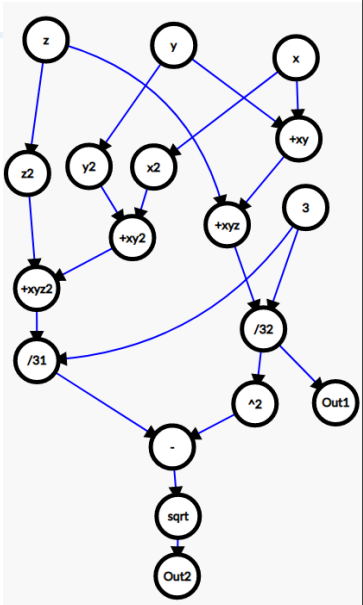
Z = (x+1) * y



Exemplul de cod 2

- Function Med_Dev calculeaza media si deviatia standard a trei variabile

```
function Med_Dev(x,y,z: real
  returns real,real);
 let
  Out1 := (x + y + z)/3;
  Out2 := SQRT((x**2 + y**2 +
            z**2)/3 - Out1**2);
 in
  Out1, Out2
 endlet
endfun
```



Caracteristici ale grafului asociat fluxului de date

- Funcționalitate:
  - Evaluarea unui graf de flux de date este echivalentă cu evaluarea funcției matematice corespunzătoare
- Compozibilitate:
  - Grafurile fluxului de date pot fi combinate pentru a forma noi grafuri
- Tipuri de structuri Data Flow
  - Static: dependentele de date sunt stabilite la compilare
  - Dinamic: dependentele de date sunt stabilite la rulare

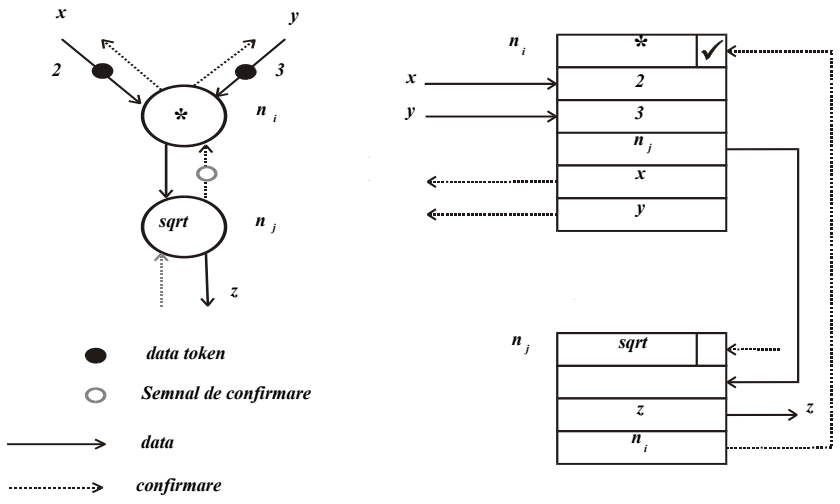


Data Flow Static

- Un graf Data Flow (DF) este reprezentat ca o colecție de șabloane de activitate conținând:
  - Codul instrucțiunii reprezentate **op**: `LHS = op (RHS1, RHS2, …)`
  - Operanzii instructiunii (**RHS**s)
  - Destinația operatiei (**LHS**) – sloturile din șabloanele de activitate din secvente care primesc valoarea rezultatului operatiei
- Deficiente Data Flow static:
  - Iterațiile consecutive ale unei bucle pot fi executate doar sub forma de pipeline
  - Datorită token-urilor de confirmare, traficul acestora este dublat
  - Lipsa suportului de programare pentru limbaje moderne
  - Lipsa apelurilor de procedură sau a recursivitatii
- Avantaj Data Flow static:
  - Este un model simplu de implementat



Data Flow – Exemplu de Activitate



Data Flow Dinamic

- Fiecare iterație de buclă sau invocare de subprogram se poate executa în paralel ca o instanță separată a unui subgraf reentrant
- Replicarea este doar conceptuală
- Fiecare **token** are un tag cu
  - Adresa instrucțiunii pentru care este destinată valoarea datelor
  - Informații de context
- Fiecare arc poate fi văzut ca o multime care poate conține un număr arbitrar de token-uri cu tag-uri diferite
- Regula de activare și declanșare este:
  - Un nod este activat și declanșat de îndată ce tokenurile cu etichete identice sunt prezente pe toate arcele de intrare
- Restrictiile structurale sunt ignorate!

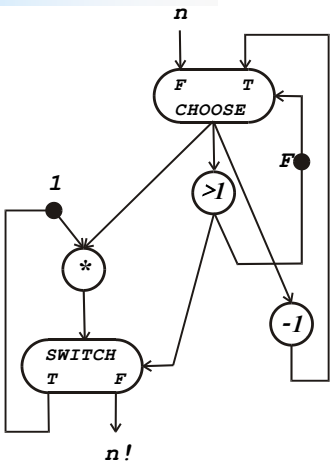
Data Flow Static vs. Dinamic

Caracteristica	Static Data Flow	Dynamic Data Flow
Dependenta date	Determinat la compilare	Determinat la runtime
Flexibilitate	Putin flexibil	Mai flexibil
Complexitate	Putin complex	Mai complex



Exemplu – Calculul Factorial == n!

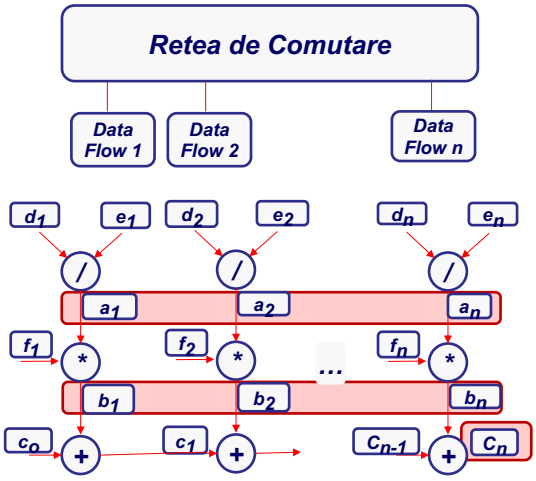
```
initial j = n; k = 1
 while j > 1 do
   j = j - 1;
  k = k * j;
return k
```





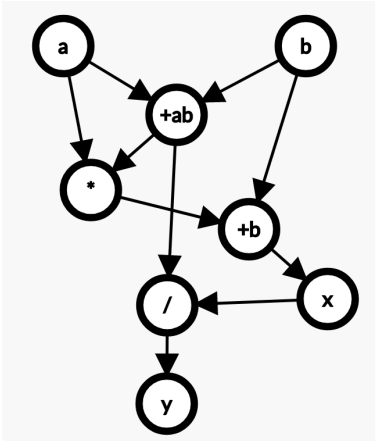
Exemplu de prelucrare vectori

```
Input d,e,f
//a,b,c,d,e,f
//vectori de date
 C0=0
For i=1 to n
    ai=di / ei
    bi=ai*fi
    ci=bi+ci-1
End for
Output a,b,cn
```



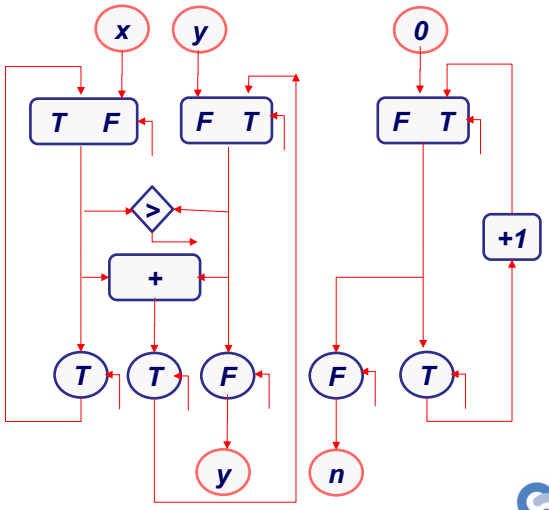
Ordinea instructiunilor este irelevanta

```
input a,b
 y = (a+b)/x
 x = (a*(a+b))+b
output y,x
```



Exemplu de bucla

```
input y,x
 n = 0
 while y<x do
    y =  y + x
    n =  n + 1
 end
output y,n
```



Caracteristici de baza

- Executia instructiunilor este bazata pe disponibilitatea datelor
- Datele sunt pastrate in cadrul instructiunii
- Disponibilitatea datelor este verificata de o unitate specializata
- Tokenul associat datelor este verificat de o unitate specializata
- Executia instructiunilor este asincrona

314

Avantaje si Dezavantaje

Avantaje

- Potential ridicat de parallelism
- Viteza crescuta de executie
- Usor de integrat comunicatia si sincronizarea

Dezavantaje

- Overhead mare pentru control
- Manipularea dificila a structurilor de date



314

315

Unde se foloseste Data Flow in Procesoarele Actuale?

- Vector Processing Units (VPUs) – Procesare SIMD
  - VPU-urile sunt procesoare specializate pentru procesarea vectoriala
  - Domenii aplicative: ML, procesare multimedia
  - Utilizeaza DF pentru executia eficienta a operatiilor vectoriale
  - Tehnologii: Intel Movidius Myriad X VPU – NNs in Edge Computing Apps
- Graphics Processing Units (GPUs)
  - GPU-urile sunt procesoare specializare pentru taskuri de procesare grafica
  - Domenii aplicative: optimizarea deplasarii de date si a executiei in SM-uri
  - Tehnologii: GPU-urile NVIDIA folosesc DF pentru rezolvarea dependentelor de date si pentru schedulingul de instructiuni
- Domain-Specific Accelerators (DSAs)
  - DSA-urile sunt procesoare specializate pentru anumite domenii aplicative
  - Domenii aplicative: procesare de semnal, criptografie
  - Utilizeaza DF pentru performante crescute si cresterea eficientei energetice
  - Tehnologii: Google TPU folosesc DF pentru executia eficienta a operatiilor de tip tensor



315

Unde se foloseste Data Flow in Procesoarele Actuale?

- Neural Processing Units (NPUs)
  - NPU-urile = procesoare specializate pentru accelerare workloaduri AI
  - Domenii aplicative: NNs (CNN, DNN, GANS, etc.)
  - Utilizeaza DF la optimizarea deplasarii de date in arhitecturi paralele
  - Tehnologii: Graphcore Colossus NPU folosesc abordari inspirate de DF pentru a gestiona dependentele de date si pentru a imbunatati performantele NNs
- Application-Specific Integrated Circuits (ASICs)
  - ASIC-urile sunt cipuri dedicate unor anumite tipuri de aplicatii ce ofera performanta si eficienta energetica
  - Domenii aplicative: DSP, NN, AI, ML
  - Tehnologii: Amazon Inferentia NPU sunt dezvoltate pentru a realiza inferenta in cadrul unor sisteme ML.
  - Utilizeaza DF la optimizarea traficului de date si al executiei fluxului de instructiuni

